RESEARCH BLUEPRINT

CERA

Cross-Execution Recovery through Effect Adoption for Stochastic Agents

SOSP 2027

Core thesis. When an agent has already externalized an effect and recovery produces a causally different execution, the system should not be forced to choose between blindly replaying the old execution and discarding all historical work. CERA treats realized effects as immutable history and determines which effects remain semantically compatible with the recovered execution.

Design boundary. CERA targets the post-commit escape regime: an effect is externally visible; the original execution fails or is replaced; recovery may diverge; whole-run rollback is unsafe, unavailable, or unnecessarily destructive; some historical work may remain valid; and supported resources expose authoritative identity, version, or validation semantics. Unsupported cases fail closed. CERA neither undoes external reality nor infers arbitrary natural-language intent.

1 Problem Setting and Target Regime

Long-running agents interact with files, databases, object stores, deployment controls, messages, and other stateful resources. A fault may force recovery after effects have already escaped local containment. The recovered agent need not reproduce the original causal trajectory: model sampling, model or prompt changes, tool outputs, retrieved context, memory, environment state, or resource evolution may change subsequent decisions.

The target regime is defined by the conjunction

$$\mathsf{Escape}(H_o) \ \wedge \ \mathsf{Substitute}(X_o, X_n) \ \wedge \ \mathsf{RollbackBlocked}(X_o) \ \wedge \ \mathsf{Supported}(e). \tag{1}$$

More explicitly:

1. an external effect has become real;
2. the old execution fails or is intentionally replaced;
3. the recovered execution is causally different;
4. whole-run rollback is unsafe, unavailable, or unnecessarily destructive;
5. at least one historical effect may remain semantically valid; and
6. the relevant resource exposes authoritative identity/version evidence and a supported validation interface.

The question is therefore not merely where recovery may resume. It is whether a realized historical effect can remain part of the causal history of a different execution.

$$\boxed{\text{Given } e \in H_o \text{ and a causally different } X_n, \quad \text{when does } \mathrm{Compat}(e, X_o, X_n) \text{ hold?}} \tag{2}$$

2 Effect Adoption and Research Gap

2.1 Effect adoption

Adoption is the logical reuse of an effect that has already happened. It is distinct from replay, compensation, and physical rollback. Once externalized, the effect is immutable history. CERA establishes whether the semantic obligation created by that effect remains valid for the recovered causal execution.

Mechanism	Decision	Limitation addressed by CERA
Replay	Re-execute prior computation	Does not justify reuse when the recovered branch differs.
Recovery boundary	Choose where execution may resume	Cannot express heterogeneous per-effect validity after substitution.
Compensation	Undo or repair earlier effects	May be impossible after post-commit escape.
Selective recovery	Preserve unaffected work	Typically reasons over dependency/re-execution structure rather than relational semantic validity.
CERA adoption	Retain an already-realized effect	Effect-level, resource-aware, cross-execution semantic validity.

2.2 Precise research gap

Existing recovery systems address admissible recovery boundaries, replay/fencing, checkpoint/restore, transactional containment, staged effects, compensation, and dependency-guided selective recovery. The unresolved object is narrower:

> *a semantic relation deciding whether an already-externalized effect from one execution remains valid after replacing that execution with a causally different execution.*

The novelty claim is therefore not "semantic recovery" in general and not OS provenance itself. It is cross-execution effect compatibility backed by resource semantics and authoritative evidence.

3 Research Contributions

C1 - Cross-execution compatibility semantics. CERA makes $\mathrm{Compat}(e, X_o, X_n)$ a first-class semantic object independent of the implementation.

C2 - CERA recovery runtime. CERA combines execution provenance, authoritative resource evidence, historical-effect state, compatibility checking, and an admission boundary that prevents unsupported new effects from escaping during recovery.

C3 - Safe history reconstruction. A small preservation calculus derives sound adoption witnesses, and an exact selector computes the maximum-value safe historical subset for a restricted compositional resource class. The result is explicitly separated from boundary-only recovery.

C4 - CERA-Bench. CERA-Bench is an independent benchmark and oracle suite for post-commit stochastic recovery under execution substitution. Each case contains the old execution, fault/divergence condition, recovered execution, historical/new effects, authoritative state/provenance evidence, and independently defined compatibility labels. The benchmark is an evaluation artifact, not a source of semantic truth for CERA.

4 Formal Semantic Model

4.1 Executions

An execution is

$$X = (G, W, H, A), \tag{3}$$

where $G = (V, E, \tau)$ is a causal provenance DAG, W is the supported observable resource state, H is immutable history of realized external effects, and A is authority/capability state. An event records stable identity, causal parents, inputs/outputs, resource reads/writes, executor identity, and relevant environment information.

4.2 Resource semantics

For a supported resource class,

$$R = (S_R, \rightarrow_R, \preceq_R, \mathrm{Id}_R, \mathrm{Anchor}_R, \mathrm{Sem}_R), \tag{4}$$

where S_R is the state space, $\rightarrow_R$ is the externally visible transition relation, $\preceq_R$ is an optional monotone/refinement relation, Id_R identifies semantic objects, Anchor_R provides authoritative version/epoch evidence, and Sem_R provides invariants, validation operations, and atomicity guarantees. Resource adapters, not LLM judgments, define the trusted semantic boundary.

Supported prototype classes are a filesystem, PostgreSQL, a local S3-compatible object store, and a local transactional service.

4.3 Historical effects

A historical effect is

$$e = (\mathrm{id}, \mathrm{op}, r, s_o, w, F_e, a, p, \pi, \alpha), \tag{5}$$

where r is the resource identity, s_o is relevant old state, w is the committed resource witness, F_e is the supported footprint, a is authority, p is the semantic postcondition, π is provenance, and α is authoritative anchor/version evidence. Once externalized, e is immutable history.

4.4 Execution substitution

After failure or intentional replacement,

$$X_o = (G_o, W_o, H_o, A_o), \qquad X_n = (G_n, W_n, H_n, A_n), \tag{6}$$

with no deterministic replay assumption and generally $G_n \neq G_o$. Let Δ denote the divergent events and $\mathrm{Infl}(\Delta)$ a conservative over-approximation of resources influenced by the divergence.

4.5 Compatibility

Definition 4.1 (Cross-execution compatibility)**.** *For a supported effect class,* $\mathrm{Compat}(e, X_o, X_n)$ *holds iff every admissible continuation of* X_n*, allowed by the resource semantics and satisfying the authority/anchor obligations of* e*, preserves the semantic postcondition established by* e*.*

This is a future-preservation property, not a one-time snapshot test. Influence overlap is evidence of possible interaction, not automatic rejection.

4.6 Adoption states

State	Semantic meaning	Required action
ADOPT	A compatibility witness is derivable.	Retain the historical effect.
BLOCK	Incompatibility is established.	Exclude the effect from recovered history.
UNKNOWN	Evidence is insufficient.	Fail closed: escalate, quarantine, or reject adoption.

5 Preservation Calculus and Theory

The calculus is deliberately small and resource-aware.

Rule	Sufficient condition	Interpretation
P1 Non-interference	$F_e \cap \mathrm{Infl}(\Delta) = \varnothing$ and anchors remain valid.	Divergence cannot affect the effect footprint.
P2 Restricted identity preservation	Identity and authority/version remain valid, and the obligation is invariant under all permitted state changes.	Identity is sufficient only for the supported effect class.
P3 Monotone preservation	$s_o \preceq_R s_n$ and the postcondition is upward-closed.	Later monotone state preserves the old obligation.
P4 Atomic semantic revalidation	Atomic validate-and-adopt, CAS/version, epoch/lease, or monotonicity rules out harmful invalidation.	A stale snapshot alone never establishes adoption.

Theorem 5.1 (Preservation soundness)**.** *For the supported resource transition classes,*

$$\mathrm{Derivable}_{CERA}(\omega, e, X_o, X_n) \implies \mathrm{Compat}(e, X_o, X_n). \tag{7}$$

Proof obligation. Induction over the finite preservation derivation; the semantic definition of compatibility remains independent of the checker.

Theorem 5.2 (Strict separation from boundary-only recovery)**.** *There exists a family of executions for which no uniform recovery boundary can be admitted while a non-empty subset of historical effects is safe to retain. Any policy whose semantic output is only a cut with uniform downstream keep/discard behavior cannot represent this*

subset without an equivalent effect-level compatibility relation.

Theorem 5.3 (Exact safe-history selection for the restricted class)**.** *For a supported compositional resource family with tractable intra-group conflicts and utility $V(e) \geq 0$,*

$$A^* = \arg\max_{A \subseteq H_o} \sum_{e \in A} V(e) \quad s.t. \quad \text{Safe}(A, X_o, X_n). \tag{8}$$

CERA computes A^* exactly for the supported conflict structure and escalates unsupported interaction patterns.

Proposition 5.4 (Observation separation)**.** *There exists a workload family with identical framework-visible observations but different compatibility outcomes, while additional authoritative OS/resource observations distinguish those outcomes. Thus framework-only observation can be insufficient for sound adoption.*

6 Algorithms and Runtime Architecture

6.1 Conservative cross-execution alignment

Algorithm 1 Conservative cross-execution alignment

Require: Old graph G_o, new graph G_n, stable sites, resource anchors
Ensure: Candidate alignment M and divergence set Δ

1: Index old and new events by stable execution sites and resource anchors.
2: Generate candidate matches subject to compatible parents, resource identities, and versions.
3: Reject matches violating provenance or anchor constraints.
4: Accept only uniquely justified supported matches.
5: Mark unmatched or ambiguous events as divergence Δ.
6: **return** (M, Δ)

6.2 Influence analysis

Algorithm 2 Influence analysis

Require: Divergence set Δ, aligned provenance, resource adapters
Ensure: Conservative influence set $\text{Infl}(\Delta)$

1: Compute the provenance closure reachable from Δ.
2: Project the closure onto each supported resource class.
3: Apply the resource adapter's sound over-approximation rule.
4: **return** $\text{Infl}(\Delta)$

6.3 Historical-effect adoption

Algorithm 3 Historical-effect adoption

Require: Historical effects H_o, influence set $\text{Infl}(\Delta)$, resource evidence

for all $e \in H_o$ **do**
    Derive P1–P4 in order.
    if a compatibility witness exists **then**
        $state(e) \leftarrow$ ADOPT
    else if incompatibility is established **then**
        $state(e) \leftarrow$ BLOCK
    else
        $state(e) \leftarrow$ UNKNOWN
    end if
end for
return per-effect adoption states

6.4 Safe history selection and future-effect admission

For the supported conflict class, CERA constructs the cross-effect conflict relation, solves each independent conflict group exactly, and combines the resulting safe subsets. Unsupported interaction structures are escalated. During recovered execution, new managed effects remain isolated until capability, anchor, compatibility-with-retained-history, and atomic admission checks succeed; otherwise the effect is suppressed.

The runtime consists of six trusted components: the historical-effect ledger, provenance collector, resource adapters, compatibility engine, history selector, and future-effect admission boundary. The prototype targets Linux and uses a filesystem/process substrate, PostgreSQL, and a local S3-compatible object store. eBPF is an observational extension rather than a prerequisite for the semantic core.

7 CERA-Bench

CERA-Bench is defined as

$$B = \langle X_o, f, X_n, E_o, E_n, GT \rangle, \tag{9}$$

where f is a declared fault/divergence condition and GT is an independent oracle for historical-effect compatibility and new-effect admission. The benchmark is generated from controlled resource semantics and real tool-mediated workloads. The trusted oracle is derived from resource state transitions, identities, versions, invariants, and atomic validation semantics; it is never an LLM judgment or an invocation of the CERA checker.

The benchmark covers four workload families: software engineering, deployment/state transition, data workflows, and transactional workflows. Representative resources are the filesystem, PostgreSQL, a local object store, and a local transactional service.

8 Experimental Program

The empirical program separates the motivating phenomenon, semantic correctness, systems comparison, observation value, operating envelope, and implementation safety.

Stage	Question	Evaluation role
B0	Can exact benchmark instances with independent compatibility ground truth be generated?	Construct and freeze CERA-Bench.
E0A	Does stochastic post-commit recovery create divergent executions with mixed historical-effect validity?	Controlled reality study with exact semantics.
E0B	Do repeated real agent executions materially diverge?	External OSWorld realism check; not a semantic oracle.
E1	Does the implementation match the independent oracle and exact selector?	Semantic correctness and exact-selection validation.
E2	Does effect-level adoption improve recovery over strong baselines?	Flagship system comparison.
E3	Does authoritative OS/resource evidence change safe decisions?	Observation-separation evaluation.
E4	In which regimes does CERA preserve useful work, and when does it conservatively give up?	Externalization/divergence/opacity/conflict sweep.
E5	Is the runtime safe and practically efficient under faults and adversarial conditions?	Overhead and fail-closed safety evaluation.

9 Evaluation Metrics

For a retained set $A \subseteq H_o$, define

$$\text{InvalidRetention} = |\{e \in A : \ GT(e) = \textsf{BLOCK}\}|, \tag{10}$$

$$\text{ValidWorkRetained} = \sum_{e \in A,\ GT(e)=\textsf{ADOPT}} V(e), \tag{11}$$

$$\text{UnnecessaryDiscard} = \sum_{e \notin A,\ GT(e)=\textsf{ADOPT}} V(e), \tag{12}$$

$$\text{Coverage} = \frac{\#\textsf{ADOPT} + \#\textsf{BLOCK}}{\#\text{supported effects}}, \tag{13}$$

$$\text{RecoveryEfficiency} = \frac{\text{ValidWorkRetained}}{\text{RecoveryCost}}. \tag{14}$$

For stochastic execution divergence,

$$D_{\text{action}} = 1 - \frac{|\text{LCS}(A_o, A_n)|}{\max(|A_o|, |A_n|)}, \qquad \kappa = \frac{|\text{Cone}(\Delta)|}{|V_o| + |V_n|}. \tag{15}$$

For controlled cases,

$$\text{MixedCompatibility} = \mathbb{1}[\exists e_i : state(e_i) = \textsf{ADOPT} \ \wedge \ \exists e_j : state(e_j) = \textsf{BLOCK}], \tag{16}$$

and

$$\text{AdoptionOpportunity} = \frac{\#\textsf{ADOPT}}{\#\textsf{ADOPT} + \#\textsf{BLOCK}}, \tag{17}$$

with UNKNOWN excluded from the denominator. Binary rates use Wilson 95% confidence intervals; continuous runtime measures use median, IQR, and p95 where appropriate. Safety results are not averaged away: any unsupported invalid adoption is a failure.

10 Claims, Kill Criteria, and Submission Gate

The paper is submitted only if the following claims are supported:

1. Compat(e, X_o, X_n) is independently defined and non-tautological.
2. The preservation calculus is sound for the stated supported resource classes.
3. A real execution family contains cases in which a uniform recovery boundary cannot retain all safe historical effects while CERA retains a non-empty safe subset.
4. Strong recovery baselines cannot reproduce the central result without an equivalent effect-level semantic relation.
5. At least one real workload exhibits an observation limitation resolved by authoritative resource/OS evidence.
6. The independent oracle confirms zero invalid adoption on the supported evaluation space.
7. The useful-work advantage survives recovery-cost accounting and adversarial testing.

The project is narrowed or stopped if any of the following occur: useful partially adoptable history is negligible; a strong baseline reproduces the result without an equivalent semantic relation; compatibility collapses to ordinary dependency reachability plus arbitrary predicates; independent ground truth cannot be defined; supported effects overwhelmingly become UNKNOWN; certification cost removes the recovery benefit; or an unsupported invalid effect is adopted or a managed rejected effect escapes.

11 Guarantee Boundary

CERA does not infer arbitrary user intent, repair arbitrary irreversible reality, or reason soundly about opaque unmanaged resources. A resource is supported only when identity, authority, state/version evidence, transition semantics, and the required validation/atomicity guarantees are exposed by its adapter. Outside that boundary, the system fails closed.

12 Final Research Thesis

CERA is a recovery system for the post-commit escape regime. It treats already-realized effects as immutable history, defines when those effects remain compatible with a causally different execution, proves adoption using a small resource-aware preservation calculus, computes a maximum-value safe retained history for a restricted compositional class, and prevents unsupported new effects from escaping during recovery. CERA-Bench makes the semantic decision independently measurable, while the system evaluation establishes correctness, preservation benefit, observation value, operating limits, and safety.

Selected References

1. Yang et al., "DART: Semantic Recoverability for Structured Tool Agents," arXiv:2605.23311, 2026.
2. Zheng et al., "ACRFence: Preventing Semantic Rollback Attacks in Agent Checkpoint-Restore," arXiv:2603.20625, 2026.
3. Chen et al., "Cordon: Semantic Transactions for Tool-Using LLM Agents," arXiv:2606.17573, 2026.
4. Mohammadi et al., "Atomix: Timely, Transactional Tool Use for Reliable Agentic Workflows," arXiv:2602.14849, 2026.
5. Chang et al., "Mnemosyne: Agentic Transaction Processing for Validating and Repairing AI-generated Workflows," arXiv:2607.00269, 2026.
6. Zhuang et al., "AgentRewind: Recoverable Execution for Long-Horizon LLM Agents," arXiv:2608.14380, 2026.
7. Wu et al., "Crab: A Semantics-Aware Checkpoint/Restore Runtime for Agent Sandboxes," arXiv:2604.28138, 2026.
8. Yu et al., "From Faulty Memories to Corrected Actions: Dependency-Guided Rollback Repair for Memory-Augmented Agents," arXiv:2608.10502, 2026.
9. Zeldovich et al., "Retro: Targeted Recovery for Compromised File Systems," OSDI, 2010.
10. Goel et al., "The Taser Intrusion Recovery System," SOSP, 2005.
11. Kim et al., "Intrusion Recovery Using Selective Re-execution," OSDI, 2010.
12. Bates et al., "Trustworthy Whole-System Provenance for the Linux Kernel," USENIX Security, 2015.
13. Pasquier et al., "Practical Whole-System Provenance Capture," SoCC, 2017.
14. Liu et al., "AgentBench: Evaluating LLMs as Agents," ICLR, 2024.
15. Yang et al., "Qwen3 Technical Report," arXiv:2505.09388, 2025.